



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 06

NOMBRE COMPLETO: Rosas Martinez Francisco Javier

N° de Cuenta: 320109766

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 11/Octubre/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio 1:

Crear un dado de 8 caras y texturizarlo por medio de código.

Para resolver el ejercicio, primero tuve que crear la figura del octaedro. Posteriormente, utilicé una imagen editada en GIMP para asignar individualmente la textura correspondiente a cada una de sus caras. Finalmente, instancié la figura con una escala adecuada para lograr una correcta visualización.

Bloques de código generado

```
Texture octaedroTexture;
```

```
void CrearOctaedro()
{
    unsigned int octaedro_indices[] = {
        // Caras superiores
        0, 1, 2,    // Cara 1
        3, 4, 5,    // Cara 2
        6, 7, 8,    // Cara 3
        9, 10, 11,   // Cara 4

        // Caras inferiores
        12, 13, 14,   // Cara 5
        15, 16, 17,   // Cara 6
        18, 19, 20,   // Cara 7
        21, 22, 23    // Cara 8
    };
}
```

```

GLfloat octaedro_vertices[] = {
    // Caras superiores
    // Cara 1 top - derecha - frente
    0.0f, 0.5f, 0.0f, 0.49f, 0.74f, 0.0f, 0.707f, 0.707f, //0
    0.5f, 0.0f, 0.0f, 0.25f, 0.51f, 0.0f, 0.707f, 0.707f, //1
    0.0f, 0.0f, 0.5f, 0.02f, 0.74f, 0.0f, 0.707f, 0.707f, //2

    // Cara 2 top - frente - izquierda
    0.0f, 0.5f, 0.0f, 0.52f, 0.26f, -0.707f, 0.707f, 0.0f, //3
    0.0f, 0.0f, 0.5f, 0.74f, 0.49f, -0.707f, 0.707f, 0.0f, //4
    -0.5f, 0.0f, 0.0f, 0.97f, 0.26f, -0.707f, 0.707f, 0.0f, //5

    // Cara 3 top - izquierda - atrás
    0.0f, 0.5f, 0.0f, 0.49f, 0.27f, 0.0f, 0.707f, -0.707f, //6
    -0.5f, 0.0f, 0.0f, 0.27f, 0.49f, 0.0f, 0.707f, -0.707f, //7
    0.0f, 0.0f, -0.5f, 0.73f, 0.49f, 0.0f, 0.707f, -0.707f, //8

    // Cara 4 top - atrás - derecha
    0.0f, 0.5f, 0.0f, 0.48f, 0.76f, 0.707f, 0.707f, 0.0f, //9
    0.0f, 0.0f, -0.5f, 0.03f, 0.76f, 0.707f, 0.707f, 0.0f, //10
    0.5f, 0.0f, 0.0f, 0.25f, 0.98f, 0.707f, 0.707f, 0.0f, //11

    // Caras inferiores
    // Cara 5 bottom - frente - derecha
    0.0f, -0.5f, 0.0f, 0.51f, 0.74f, 0.0f, -0.707f, 0.707f, //12
    0.0f, 0.0f, 0.5f, 0.97f, 0.74f, 0.0f, -0.707f, 0.707f, //13
    0.5f, 0.0f, 0.0f, 0.75f, 0.51f, 0.0f, -0.707f, 0.707f, //14

    // Cara 6 bottom - izquierda - frente
    0.0f, -0.5f, 0.0f, 0.48f, 0.26f, -0.707f, -0.707f, 0.0f, //15
    -0.5f, 0.0f, 0.0f, 0.03f, 0.26f, -0.707f, -0.707f, 0.0f, //16
    0.0f, 0.0f, 0.5f, 0.25f, 0.49f, -0.707f, -0.707f, 0.0f, //17

    // Cara 7 bottom - atrás - izquierda
    0.0f, -0.5f, 0.0f, 0.48f, 0.24f, 0.0f, -0.707f, -0.707f, //18
    0.0f, 0.0f, -0.5f, 0.25f, 0.02f, 0.0f, -0.707f, -0.707f, //19
    -0.5f, 0.0f, 0.0f, 0.03f, 0.24f, 0.0f, -0.707f, -0.707f, //20

    // Cara 8 bottom - derecha - atrás
    0.0f, -0.5f, 0.0f, 0.5f, 0.74f, 0.707f, -0.707f, 0.0f, //21
    0.5f, 0.0f, 0.0f, 0.73f, 0.51f, 0.707f, -0.707f, 0.0f, //22
    0.0f, 0.0f, -0.5f, 0.27f, 0.51f, 0.707f, -0.707f, 0.0f, //23
};

Mesh* octaedro = new Mesh();
octaedro->CreateMesh(octaedro_vertices, octaedro_indices,
    24 * 8,
    24);
meshList.push_back(octaedro);
}

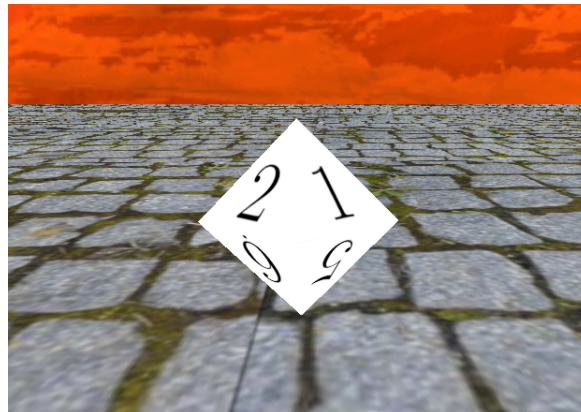
```

```
CrearOctaedro();
```

```
octaedroTexture = Texture("Textures/octaedro.png");  
octaedroTexture.LoadTextureA();
```

```
// Octaedro  
model = glm::mat4(1.0f);  
model = glm::translate(model, glm::vec3(2.0f, 3.5f, -2.0f));  
model = glm::scale(model, glm::vec3(3.0f, 3.0f, 3.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
octaedroTexture.UseTexture();  
meshList[5]->RenderMesh();
```

Código en ejecución





Ejercicio 2:

Importar el modelo de su coche con sus 4 llantas acomodadas y tener texturizadas las 4 llantas (diferenciar caucho y rin).

Para resolver el ejercicio, abrí en Blender mi modelo de la llanta de la práctica pasada. Primero separé el rin de la llanta para poder asignar una textura diferente a cada uno. Después los volví a unir y exporté el modelo en formato .obj. Finalmente, importé el modelo en OpenGL y coloqué cada una de las llantas en su posición correcta para que se visualizaran adecuadamente.

Bloques de código generado

```
Model llanta;  
Model coche;
```

```
coche = Model();  
coche.LoadModel("Models/carrop06.obj");  
  
llanta = Model();  
llanta.LoadModel("Models/llantap06.obj");
```

```

// Coche base
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 2.0f, -0.5f));
modelaux = model;
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.4f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
coche.RenderModel();

// Llanta 1
model = modelaux;
model = glm::translate(model, glm::vec3(-6.2f, -2.4f, -2.8f));
model = glm::scale(model, glm::vec3(4.2f, 4.2f, 4.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
llanta.RenderModel();

```

```

// Llanta 2
model = modelaux;
model = glm::translate(model, glm::vec3(-6.2f, -2.4f, 4.5f));
model = glm::scale(model, glm::vec3(4.2f, 4.2f, 4.2f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
llanta.RenderModel();

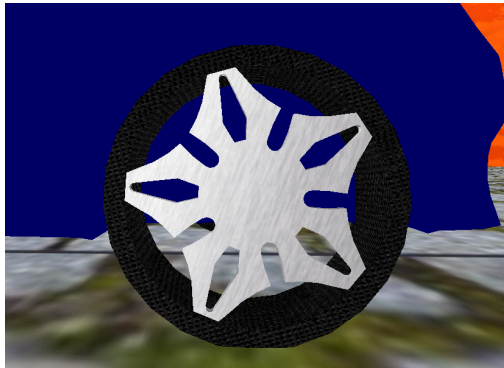
// Llanta 3
model = modelaux;
model = glm::translate(model, glm::vec3(7.4f, -2.4f, -2.8f));
model = glm::scale(model, glm::vec3(4.2f, 4.2f, 4.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
llanta.RenderModel();

// Llanta 4
model = modelaux;
model = glm::translate(model, glm::vec3(7.4f, -2.4f, 4.5f));
model = glm::scale(model, glm::vec3(4.2f, 4.2f, 4.2f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
llanta.RenderModel();

```

Código en ejecución





Ejercicio 3:

Ejercicio 3: Texturizar la cara del personaje de la imagen tipo cars en el espejo (ojos) y detalles en cofre y parrilla de su propio modelo de coche

Con este ejercicio, también tuve que separar el cofre, los espejos y varias parrillas para asignarles una textura individual en Blender. Luego exporté cada parte en formato .obj y las importé en OpenGL, asegurándome de colocarlas con la posición y orientación correctas.

Bloques de código generado

```
Model ojos;  
Model Parrilla1;  
Model Parrilla2;  
Model Parrilla3;  
Model Parrilla4;  
Model cofre;
```



```

ojos = Model();
ojos.LoadModel("Models/ojoscars.obj");

Parrilla1 = Model();
Parrilla1.LoadModel("Models/parrillaa.obj");

Parrilla2 = Model();
Parrilla2.LoadModel("Models/parrilla2.obj");

Parrilla3 = Model();
Parrilla3.LoadModel("Models/parrilla3.obj");

Parrilla4 = Model();
Parrilla4.LoadModel("Models/parrilla4.obj");

cofre = Model();
cofre.LoadModel("Models/cofre06.obj");

```

```

// Ojos
model = modelaux;
model = glm::translate(model, glm::vec3(3.4f, 2.0f, 0.8f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
ojos.RenderModel();

// Parrilla
model = modelaux;
model = glm::translate(model, glm::vec3(11.5f, -0.5f, 0.8f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
Parrilla1.RenderModel();

// Parrilla 2
model = modelaux;
model = glm::translate(model, glm::vec3(11.2f, -1.85f, 0.8f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
Parrilla2.RenderModel();

// Parrilla 3
model = modelaux;
model = glm::translate(model, glm::vec3(10.5f, -1.85f, -2.3f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
Parrilla3.RenderModel();

// Parrilla 4
model = modelaux;
model = glm::translate(model, glm::vec3(10.5f, -1.85f, 3.8f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
Parrilla4.RenderModel();

// Cofre
model = modelaux;
model = glm::translate(model, glm::vec3(8.1f, 0.95f, 0.8f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
cofre.RenderModel();

```


Código en ejecución



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

- En esta práctica se me complicó bastante el uso correcto de las texturas. Al importar los modelos a OpenGL, se cargaban sin texturas. Después descubrí que el problema era que estaba aplicando la imagen como imagen de entorno en lugar de asignarla correctamente como textura del material.
- Además, tuve dificultades para colocar las texturas de forma adecuada, ya que las imágenes aparecían distorsionadas sobre los modelos. No logré resolverlo hasta que volví a ver el video proporcionado por el profesor, donde comprendí cómo corregir mi error y aplicar correctamente las texturas.

Conclusión:

Al finalizar esta práctica logré aprender y aplicar correctamente el uso de texturas en mis modelos, tanto desde Blender como mediante código en OpenGL. Aunque fue una práctica que me tomó bastante tiempo debido a las complicaciones que tuve, resultó muy entretenida. Gracias a los conocimientos adquiridos, podré avanzar en mi proyecto final.

Bibliografía :

OpenGL Architecture Review Board, Shreiner, D., Woo, M., Neider, J., & Davis, T. (2010). OpenGL Programming Guide: The Official Guide to Learning OpenGL (7th ed.). Addison-Wesley.

OpenGL. (s. f.). OpenGL Documentation. Khronos Group. Recuperado de <https://www.khronos.org/opengl/>

Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3 (8th ed.). Addison-Wesley Professional.